Jac DB and WeBridge: TPC-C + littleX

Measured SQL round-trip cost and query fusion

Research problem: which database operations can each system combine across a complete application request?

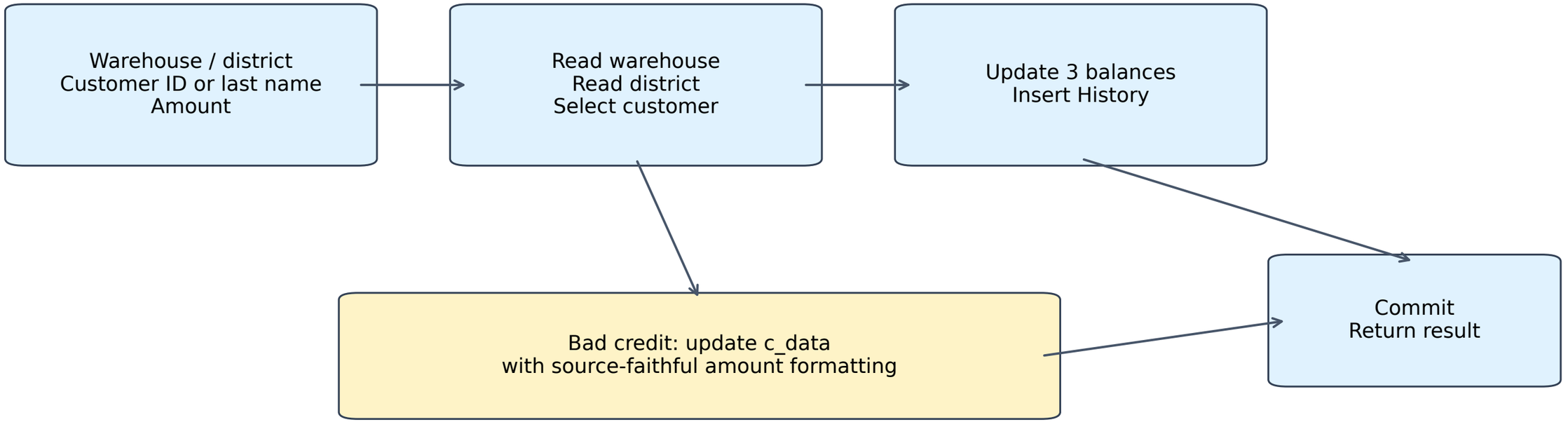
Compare the colleague's transactional and social-network workloads using native Jac and three independently checked Java forms, each paired with actual WeBridge compilation.

Compare summed SQL round-trip time and command counts from the same requests, then use executed SQL and stored-program profiles to explain the work that remains.

Source: workload-contracts.json; coverage-contract.json; runtime-provenance.json

Example: one Payment updates four business relations

TPC-C: identify a customer, update balances, append History, commit atomically



Source: tpcc-plan-900.json; workload-contracts.json; verify_tpcc_model.py

Three Java forms isolate different representation costs

Each direct / WeBridge pair uses the same application source, SQL and MySQL fixture

Form	Database representation and request behavior	What the pair measures
Full	Graph anchors, types, access checks, hydration and mutations; matches the measured warm native access sequence	WeBridge's ability to combine the full access path
Pruned	Same business results with explicitly listed graph-framework work removed	WeBridge after reducing graph-framework overhead
Conventional	Direct relational tables and source-faithful business reads / writes; littleX feeds use set-oriented SQL	WeBridge on a conventional SQL application
Native Jac	PostgreSQL 18, original and final runtime source trees	Effect of the Jac changes on the original workload

Source: tpcc/TpccSql.java; tpcc/TpccGraph.java; littlex/LittleXSql.java; littlex/LittleXFull.java; littlex/LittleXPruned.java; native/REPORT.md

Example solution: Payment's hot path becomes one CALL

Accepted conventional, good-credit, last-name Payment; separate +2 ms instrumented profile

Observed location	Direct Java	Actual WeBridge procedure
Complete wire synchronization cycles	9 / 9 / 9 measured requests	1 / 1 / 1 measured requests
Business reads and writes	3 SELECTs + 4 writes sent separately	3 cursor SELECTs + 4 writes inside CALL
Transaction controls	2 SET autocommit commands on wire	2 transaction SET statements inside CALL
Generated procedure work	Application issues direct SQL	4 SAVEPOINTs; 4 metadata/result SELECTs; 4 scalar returns
Correctness	13 evolving request results and retained affected rows	Matches direct results, 8 relation snapshots and global counts

Source: profiles/tpcc-final-payment-name-GC-sp-1792543948055566958/paired-results-and-state.json; corresponding baseline / SP analysis.json

Workloads preserve the colleague's original behavior

Frozen inputs and independent result / state models precede timing

Workload	Original reference	Latency matrix
TPC-C	Seed-42 warehouse; seed-4242 plan; 900 requests including 7 real invalid-item rollbacks	7 representative requests; 10 warmup + 30 measured repetitions; fresh initial fixture per cell
littleX W1 / W2	100 users; 5,000 follows; 20 / 50 / 100 / 200 tweets per user; full feed and 3 writes	16 workloads; original deterministic targets; 10 warmup + 30 measured requests
littleX W3	Paged feed mixed with 0 / 5 / 15 / 30% writes; original 110 warmup + 1,000 measured schedule	4 workloads; 110 warmup + 100 measured requests; one complete original 100-slot schedule
Coverage	8 configurations per workload	27 workloads × 8 configurations × 3 delay settings = 648 cells

Source: workload-contracts.json; tpcc-plan-900.json; littlex-schedules.json; coverage-contract.json

Primary timing measures complete SQL round trips

Frontend command arrival → complete response delivery; summed per application request

Compact protocol collectors record complete exchanges at 0 / 2 / 10 ms added database-response delay. Every primary request supplies both its summed SQL round-trip time and its actual command counts.

The boundary includes database execution and response transfer. Application computation between calls stays outside. All request-owned connections contribute, including native identity lookups; a WeBridge CALL is counted once.

Compilers, profilers and timing runs share an exclusive host lease. Complete returned results and final committed state gate the measured values. Separate deep profiles retain nested SQL; application and database-server timings retain their own scopes.

Source: RTT_MEASUREMENT_PLAN.md; METHODS.md; compact collector sources and per-request RTT records

RTT and SQL counts use the same measured requests

Original workload schedules and explicit initialization / observer boundaries

Measurement	Request scope
TPC-C latency cells	Ten mutating warmups + thirty measured repetitions per representative transaction, arm and delay.
littleX pure-operation latency	Ten warmups + thirty measured requests per tier, operation, arm and delay.
littleX mixed latency	110 warmups + a complete 100-slot measured schedule at each write ratio.
Original-schedule references	TPC-C: 100 warmup + 800 measured. littleX W1/W2: 110 + 500; W3: 110 + 1,000.
SQL-work attribution	Separate instrumented profiles identify internal statements, transactions, runtime metadata and fallback. Their actual sample windows remain stated.

Source: coverage-contract.json; RTT_MEASUREMENT_PLAN.md; normalized-rtt source files; rtt-cycle-evidence.json

Actual WeBridge results include integration provenance

Record → native replay → synthesis → install → active execution → result / state gate

Every compiled result identifies the original frontend outcome, applied compatibility changes, captured transaction boundary and immutable source / class hashes.

Final captures use the SDK's JDBC transaction boundary. MySQL sessions and procedure creation retain strict modes and add NO_UNSIGNED_SUBTRACTION; global SQL mode stays unchanged.

Stage failures retain the generated artifacts and exact error. Procedure timing, activation and fallback come from executed requests and wire traces.

Source: tpcc/case-status.json; littlex/cases; compatibility sources, tests, build manifests and stage logs

Representative results: DB-call RTT

Median per-request sum (ms), zero added delay; baseline → optimized

Workload	Jac: unoptimized → optimized	Jac-like Java: direct → WeBridge	Original SQL Java: direct → WeBridge
TPC-C Payment, name / GC	3.98 → 4.20	52.44 → replay failed	3.84 → 3.23
TPC-C NewOrder, 15 lines	7.25 → 6.96	28.88 → replay failed	13.49 → synthesis failed
littleX full feed, 20 tweets/user	13.64 → 13.17	74.16 → synthesis failed	9.52 → 9.18
littleX full feed, 200 tweets/user	63.80 → 62.82	401.73 → synthesis failed	81.53 → 81.10
littleX like, 200 tweets/user	3.17 → 2.99	40.34 → replay failed	3.41 → 2.88
littleX create, 200 tweets/user	2.92 → 2.67	40.24 → replay failed	3.48 → 3.16
littleX mixed, 15% writes	4.85 → 4.54	188.45 → synthesis failed	34.49 → 34.24

Each arrow compares two executions of the same workload within one system and database. Jac uses PostgreSQL 18; Java and WeBridge use MySQL 5.7.

CALL counts as one client command; nested SQL work is examined separately.

Source: results.json; exact count distributions, all 27 cases and 0 / +2 / +10 ms appear in the following pages and raw evidence.

Representative results: SQL command counts

Mean commands per request [observed range], from the same zero-delay RTT requests; baseline → optimized

Workload	Jac: unoptimized → optimized	Jac-like Java: direct → WeBridge	Original SQL Java: direct → WeBridge
TPC-C Payment, name / GC	12.00 → 12.00	12.00 → replay failed	9.00 → 1.00
TPC-C NewOrder, 15 lines	14.00 → 14.00	14.00 → replay failed	68.00 → synthesis failed
littleX full feed, 20 tweets/user	18.67 [18–19] → 14.67 [14–15]	20.00 → synthesis failed	5.00 → 3.00
littleX full feed, 200 tweets/user	18.73 [18–20] → 14.73 [14–16]	20.00 → synthesis failed	5.00 → 3.00
littleX like, 200 tweets/user	11.63 [10–12] → 10.63 [9–11]	11.97 [11–12] → replay failed	6.00 → 3.00
littleX create, 200 tweets/user	9.67 [9–10] → 8.67 [8–9]	10.00 → replay failed	6.00 → 3.00
littleX mixed, 15% writes	16.76 [9–18] → 13.21 [8–14]	18.61 [10–20] → synthesis failed	5.14 [5–6] → 3.00

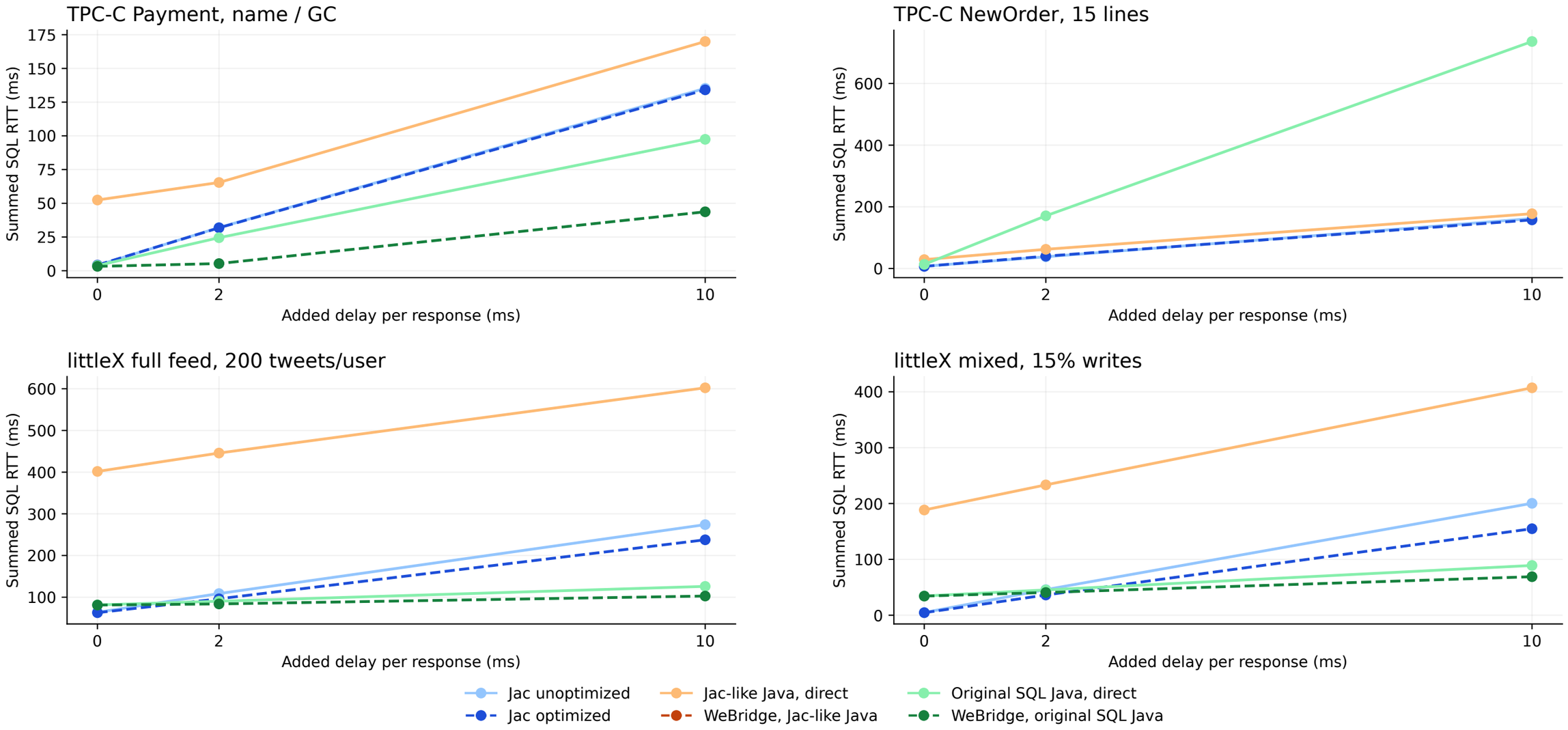
Each arrow compares two executions of the same workload within one system and database. Jac uses PostgreSQL 18; Java and WeBridge use MySQL 5.7.

CALL counts as one client command; nested SQL work is examined separately.

Source: results.json; exact count distributions, all 27 cases and 0 / +2 / +10 ms appear in the following pages and raw evidence.

How added response delay changes DB-call RTT

Median summed RTT per request; matched workload inputs at 0 / +2 / +10 ms



Source: results.json. Jac: PostgreSQL 18. Java / WeBridge: MySQL 5.7. Missing procedure curves have compiler-stage receipts in the case tables.

Complete SQL round-trip timing coverage by configuration

27 workloads × 3 delays; each compiler failure supplies stage evidence to its three delay cells

Configuration	Accepted cells	Stage-failure cells	Total cells
Jac unoptimized	81	0	81
Jac optimized	81	0	81
Jac-like Java, direct	81	0	81
WeBridge, Jac-like Java	0	81	81
Java pruned	81	0	81
WeBridge pruned	24	57	81
Original SQL Java, direct	81	0	81
WeBridge, original SQL Java	66	15	81

Source: results.json; accepted values pass result and committed-state gates

Original-schedule references complement the latency cells

TPC-C: 100 warmup + 800 measured; littleX retains the original W1 / W2 / W3 schedule lengths

Configuration	TPC-C original900	littleX accepted / 20	littleX failed stage / state
Jac unoptimized	PASS	20	0
Jac optimized	PASS	20	0
Jac-like Java, direct	PASS	20	0
WeBridge, Jac-like Java	source fallback; 0 CALLs	0	20
Java pruned	PASS	20	0
WeBridge pruned	source fallback; 0 CALLs	8	12
Original SQL Java, direct	PASS	20	0
WeBridge, original SQL Java	state rejected; 406 CALLs	20	0

Source: rtt-reference-results.json; compiler-unavailable references retain their actual stage evidence

TPC-C: payment name GC

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.98	31.99	135.23	12 / 12 / 12
Jac optimized	4.20	31.81	134.12	12 / 12 / 12
Jac-like Java, direct	52.44	65.42	169.97	12 / 12 / 12
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	3.84	24.48	97.42	9 / 9 / 9
WeBridge, original SQL Java	3.23	5.30	43.69	1 / 1 / 1

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-payment_name_GC; per-cell raw and gate links retained

TPC-C: new order rollback

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	5.63	47.94	199.73	17 (18) / 17 (18) / 17 (18)
Jac optimized	5.76	52.00	196.89	17 (18) / 17 (18) / 17 (18)
Jac-like Java, direct	31.64	74.19	218.08	17 / 17 / 17
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	11.60	144.70	628.46	58 / 58 / 58
WeBridge, original SQL Java	synthesis failed	synthesis failed	synthesis failed	stage failure

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-new_order_rollback; per-cell raw and gate links retained

TPC-C: payment id GC

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	5.25	26.25	112.07	10 / 10 / 10
Jac optimized	3.05	31.63	110.37	10 / 10 / 10
Jac-like Java, direct	7.01	29.61	113.89	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	3.76	24.64	99.33	9 / 9 / 9
WeBridge, original SQL Java	3.17	5.24	14.41	1 / 1 / 1

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-payment_id_GC; per-cell raw and gate links retained

TPC-C: new order 5

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.55	45.94	158.34	14 / 14 / 14
Jac optimized	5.21	37.97	160.02	14 / 14 / 14
Jac-like Java, direct	19.15	50.43	169.89	14 / 14 / 14
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	6.87	75.45	305.82	28 / 28 / 28
WeBridge, original SQL Java	synthesis failed	synthesis failed	synthesis failed	stage failure

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-new_order_5; per-cell raw and gate links retained

TPC-C: new order 15

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	7.25	38.06	162.05	14 / 14 / 14
Jac optimized	6.96	40.03	157.66	14 / 14 / 14
Jac-like Java, direct	28.88	62.30	177.69	14 / 14 / 14
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	13.49	170.89	736.12	68 / 68 / 68
WeBridge, original SQL Java	synthesis failed	synthesis failed	synthesis failed	stage failure

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-new_order_15; per-cell raw and gate links retained

TPC-C: payment name BC

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.91	52.60	125.68	11 / 11 / 11
Jac optimized	3.97	30.10	122.56	11 / 11 / 11
Jac-like Java, direct	34.65	58.60	154.33	11 / 11 / 11
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	4.51	26.91	110.48	10 / 10 / 10
WeBridge, original SQL Java	install failed	install failed	install failed	stage failure

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-payment_name_BC; per-cell raw and gate links retained

TPC-C: payment id BC

Median summed SQL round-trip time per request (ms); added database-response delay by column

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.60	27.59	112.90	10 / 10 / 10
Jac optimized	3.45	25.62	111.04	10 / 10 / 10
Jac-like Java, direct	6.72	29.50	112.81	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	stage failure
Original SQL Java, direct	3.75	27.14	109.37	10 / 10 / 10
WeBridge, original SQL Java	install failed	install failed	install failed	stage failure

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=tpcc-payment_id_BC; per-cell raw and gate links retained

littleX: feed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 20 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	13.64	59.90	227.77	18–19 / 18–19 / 18–20
Jac optimized	13.17	46.50	173.63	14–15 / 14–15 / 14–15
Jac-like Java, direct	74.16	116.98	295.53	20 / 20 / 20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	9.52	18.08	59.96	5 / 5 / 5
WeBridge, original SQL Java	9.18 CALL 100%	13.20 CALL 100%	38.95 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-feed-t20; per-cell raw and gate links retained

littleX: like

Median summed SQL round-trip time per request (ms); added database-response delay by column; 20 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.06	29.91	133.07	10–12 / 10–12 / 10–12
Jac optimized	2.88	27.60	117.89	9–11 / 9–11 / 9–11
Jac-like Java, direct	15.40	43.11	144.67	11–12 / 11–12 / 11–12
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.61	16.95	66.87	6 / 6 / 6
WeBridge, original SQL Java	3.03 CALL 100%	9.93 CALL 100%	34.65 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-like-t20; per-cell raw and gate links retained

littleX: follow

Median summed SQL round-trip time per request (ms); added database-response delay by column; 20 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.04	33.41	139.40	12–13 / 12–13 / 12–13
Jac optimized	3.69	31.62	129.33	11–12 / 11–12 / 11–12
Jac-like Java, direct	24.86	57.40	163.72	13 / 13 / 13
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.56	14.38	55.14	5 / 5 / 5
WeBridge, original SQL Java	3.04 CALL 100%	10.18 CALL 100%	34.48 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-follow-t20; per-cell raw and gate links retained

littleX: create tweet

Median summed SQL round-trip time per request (ms); added database-response delay by column; 20 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	2.89	24.49	105.20	9–10 / 9–10 / 9–10
Jac optimized	2.82	22.83	96.24	8–9 / 8–9 / 8–9
Jac-like Java, direct	15.17	38.44	121.26	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.30	45.99	65.65	6 / 6 / 6
WeBridge, original SQL Java	3.71 CALL 100%	9.89 CALL 100%	34.45 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-create_tweet-t20; per-cell raw and gate links retained

littleX: feed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 50 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	21.52	68.76	235.20	18–19 / 18–20 / 18–20
Jac optimized	21.47	56.60	188.10	14–15 / 14–16 / 14–16
Jac-like Java, direct	198.16	170.44	350.69	20 / 20 / 20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	21.51	29.93	72.40	5 / 5 / 5
WeBridge, original SQL Java	20.92 CALL 100%	25.17 CALL 100%	49.34 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-feed-t50; per-cell raw and gate links retained

littleX: like

Median summed SQL round-trip time per request (ms); added database-response delay by column; 50 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.16	30.55	131.27	10–12 / 10–12 / 10–12
Jac optimized	2.96	27.62	118.16	9–11 / 9–11 / 9–11
Jac-like Java, direct	19.48	46.81	145.23	11–12 / 11–12 / 11–12
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.22	17.12	65.68	6 / 6 / 6
WeBridge, original SQL Java	34.27 CALL 100%	9.94 CALL 100%	34.28 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-like-t50; per-cell raw and gate links retained

littleX: follow

Median summed SQL round-trip time per request (ms); added database-response delay by column; 50 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.81	32.91	142.93	12–13 / 12–13 / 12–13
Jac optimized	3.51	31.45	131.70	11–12 / 11–12 / 11–12
Jac-like Java, direct	33.66	61.80	177.44	13 / 13 / 13
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.19	14.46	55.35	5 / 5 / 5
WeBridge, original SQL Java	2.88 CALL 100%	9.72 CALL 100%	34.31 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-follow-t50; per-cell raw and gate links retained

littleX: create tweet

Median summed SQL round-trip time per request (ms); added database-response delay by column; 50 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	2.88	24.91	108.33	9–10 / 9–10 / 9–10
Jac optimized	2.65	23.65	96.94	8–9 / 8–9 / 8–9
Jac-like Java, direct	19.20	43.99	125.05	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.62	17.07	66.36	6 / 6 / 6
WeBridge, original SQL Java	2.99 CALL 100%	10.19 CALL 100%	34.92 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-create_tweet-t50; per-cell raw and gate links retained

littleX: feed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 100 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	34.76	77.99	246.86	18–20 / 18–20 / 18–20
Jac optimized	34.35	68.04	203.00	14–16 / 14–16 / 14–16
Jac-like Java, direct	212.91	261.64	429.54	20 / 20 / 20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	41.52	49.95	85.21	5 / 5 / 5
WeBridge, original SQL Java	40.65 CALL 100%	44.77 CALL 100%	62.96 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-feed-t100; per-cell raw and gate links retained

littleX: like

Median summed SQL round-trip time per request (ms); added database-response delay by column; 100 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.15	30.09	130.74	10–12 / 10–12 / 10–12
Jac optimized	2.95	29.41	116.90	9–11 / 9–11 / 9–11
Jac-like Java, direct	26.28	53.52	156.50	11–12 / 11–12 / 11–12
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.35	16.78	65.71	6 / 6 / 6
WeBridge, original SQL Java	3.38 CALL 100%	10.23 CALL 100%	34.85 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-like-t100; per-cell raw and gate links retained

littleX: follow

Median summed SQL round-trip time per request (ms); added database-response delay by column; 100 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.74	34.21	141.15	12–13 / 12–13 / 12–13
Jac optimized	3.84	31.27	128.83	11–12 / 11–12 / 11–12
Jac-like Java, direct	47.13	74.97	201.74	13 / 13 / 13
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	2.85	14.62	55.47	5 / 5 / 5
WeBridge, original SQL Java	3.40 CALL 100%	9.81 CALL 100%	35.15 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-follow-t100; per-cell raw and gate links retained

littleX: create tweet

Median summed SQL round-trip time per request (ms); added database-response delay by column; 100 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	2.85	24.66	106.72	9–10 / 9–10 / 9–10
Jac optimized	2.73	24.20	97.04	8–9 / 8–9 / 8–9
Jac-like Java, direct	26.34	48.23	137.45	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.43	17.12	65.89	6 / 6 / 6
WeBridge, original SQL Java	3.24 CALL 100%	9.81 CALL 100%	34.64 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-create_tweet-t100; per-cell raw and gate links retained

littleX: feed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	63.80	108.69	274.09	18–20 / 18–20 / 18–20
Jac optimized	62.82	96.19	237.70	14–16 / 14–16 / 14–16
Jac-like Java, direct	401.73	445.80	602.33	20 / 20 / 20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	81.53	90.31	125.95	5 / 5 / 5
WeBridge, original SQL Java	81.10 CALL 100%	83.84 CALL 100%	102.76 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-feed-t200; per-cell raw and gate links retained

littleX: like

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	3.17	30.65	128.32	10–12 / 10–12 / 10–12
Jac optimized	2.99	29.60	118.69	9–11 / 9–11 / 9–11
Jac-like Java, direct	40.34	68.35	171.35	11–12 / 11–12 / 11–12
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.41	17.09	66.31	6 / 6 / 6
WeBridge, original SQL Java	2.88 CALL 100%	9.64 CALL 100%	34.65 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-like-t200; per-cell raw and gate links retained

littleX: follow

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.08	33.48	143.45	12–13 / 12–13 / 12–13
Jac optimized	3.90	32.28	129.76	11–12 / 11–12 / 11–12
Jac-like Java, direct	75.29	103.20	232.99	13 / 13 / 13
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.68	14.46	55.42	5 / 5 / 5
WeBridge, original SQL Java	3.12 CALL 100%	9.43 CALL 100%	34.52 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-follow-t200; per-cell raw and gate links retained

littleX: create tweet

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	2.92	26.04	106.58	9–10 / 9–10 / 9–10
Jac optimized	2.67	22.75	97.24	8–9 / 8–9 / 8–9
Jac-like Java, direct	40.24	62.55	152.54	10 / 10 / 10
WeBridge, Jac-like Java	replay failed	replay failed	replay failed	replay
Original SQL Java, direct	3.48	17.28	66.00	6 / 6 / 6
WeBridge, original SQL Java	3.16 CALL 100%	9.71 CALL 100%	34.62 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-create_tweet-t200; per-cell raw and gate links retained

littleX: mixed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user; 0% writes

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	5.34	47.30	203.10	18 / 18–19 / 18–19 (18–20)
Jac optimized	4.85	37.21	158.18	14–15 / 14–15 / 14–15 (14–16)
Jac-like Java, direct	188.90	233.80	409.59	20 / 20 / 20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	34.49	45.82	101.04	5 / 5 / 5
WeBridge, original SQL Java	34.43 CALL 100%	41.16 CALL 100%	71.30 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-mixed-t200-w0; per-cell raw and gate links retained

littleX: mixed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user; 5% writes

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.87	47.43	200.21	9–18 / 9–19 / 9–19 (9–20)
Jac optimized	4.98	36.87	158.10	8–30 (8–30) / 8–15 (8–15) / 8–15 (8–16)
Jac-like Java, direct	189.37	230.58	413.21	10–20 / 10–20 / 10–20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	34.55	45.64	98.78	5–6 / 5–6 / 5–6
WeBridge, original SQL Java	34.47 CALL 100%	40.80 CALL 100%	70.00 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-mixed-t200-w5; per-cell raw and gate links retained

littleX: mixed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user; 15% writes

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.85	45.85	200.38	9–18 / 9–19 / 9–19 (9–20)
Jac optimized	4.54	36.23	154.89	8–14 / 8–15 / 8–15 (8–16)
Jac-like Java, direct	188.45	233.39	407.20	10–20 / 10–20 / 10–20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	34.49	45.31	89.11	5–6 / 5–6 / 5–6
WeBridge, original SQL Java	34.24 CALL 100%	40.72 CALL 100%	68.96 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-mixed-t200-w15; per-cell raw and gate links retained

littleX: mixed

Median summed SQL round-trip time per request (ms); added database-response delay by column; 200 tweets / user; 30% writes

Configuration	0 ms	+2 ms	+10 ms	SQL commands / exchanges: 0 / +2 / +10
Jac unoptimized	4.59	45.06	194.28	9–18 / 9–19 / 9–19 (9–20)
Jac optimized	4.32	35.37	155.28	8–14 / 8–15 / 8–15 (8–16)
Jac-like Java, direct	188.73	230.50	404.03	10–20 / 10–20 / 10–20
WeBridge, Jac-like Java	synthesis failed	synthesis failed	synthesis failed	synth
Original SQL Java, direct	34.30	45.33	87.37	5–6 / 5–6 / 5–6
WeBridge, original SQL Java	34.38 CALL 100%	40.60 CALL 100%	67.16 CALL 100%	3 / 3 / 3

Compact proxy: frontend command arrival to complete-response delivery; per-request sum across owned connections. SQL counts use the same requests. Database, communication and configured delay are included; application work between calls is outside. PostgreSQL 18 / MySQL 5.7. CALL %: requests invoking a procedure; E%: SDK enabled after request, shown when different. Per-request SQL/fallback and stage evidence are retained.

Source: results.json / results.csv, case_id=littlex-mixed-t200-w30; per-cell raw and gate links retained

Fresh RTT attribution: TPC-C Payment, name / GC

Primary medians by delay; dominant SQL category measured at zero added delay

Configuration	Median 0 ms	Median +10 ms	Dominant category: mean RTT	Share of mean total
Jac unoptimized	3.98	135.23	COMMIT 1.635 ms / request	40.5%
Jac optimized	4.20	134.12	COMMIT 1.585 ms / request	37.2%
Jac-like Java, direct	52.44	169.97	Graph SELECT 37.544 ms / request	56.4%
WeBridge, Jac-like Java	replay failed	replay failed	Stage receipt	—
Original SQL Java, direct	3.84	97.42	Transaction / session setting 2.646 ms / request	68.1%
WeBridge, original SQL Java	3.23	43.69	CALL: complete outer response 3.352 ms / request	100.0%

Category costs are sums within each request; their means add to the mean total. Primary medians are computed from complete per-request sums. A CALL includes its full response and internal execution.

Source: profiles/primary-rtt-attribution/final/attribution.json; exact SQL hashes, per-request reconciliation and classifier source hashes retained.

Fresh RTT attribution: littleX full feed, 20 tweets/user

Primary medians by delay; dominant SQL category measured at zero added delay

Configuration	Median 0 ms	Median +10 ms	Dominant category: mean RTT	Share of mean total
Jac unoptimized	13.64	227.77	2-hop traversal + access checks 8.736 ms / request	64.2%
Jac optimized	13.17	173.63	2-hop traversal + access checks 8.755 ms / request	66.3%
Jac-like Java, direct	74.16	295.53	2-hop traversal + access checks 41.972 ms / request	54.8%
WeBridge, Jac-like Java	synthesis failed	synthesis failed	Stage receipt	—
Original SQL Java, direct	9.52	59.96	relational lx tweets select 8.689 ms / request	90.0%
WeBridge, original SQL Java	9.18	38.95	CALL: complete outer response 9.197 ms / request	94.5%

Category costs are sums within each request; their means add to the mean total. Primary medians are computed from complete per-request sums. A CALL includes its full response and internal execution.

Source: profiles/primary-rtt-attribution/final/attribution.json; exact SQL hashes, per-request reconciliation and classifier source hashes retained.

Fresh RTT attribution: littleX full feed, 200 tweets/user

Primary medians by delay; dominant SQL category measured at zero added delay

Configuration	Median 0 ms	Median +10 ms	Dominant category: mean RTT	Share of mean total
Jac unoptimized	63.80	274.09	2-hop traversal + access checks 57.340 ms / request	89.3%
Jac optimized	62.82	237.70	2-hop traversal + access checks 57.179 ms / request	90.2%
Jac-like Java, direct	401.73	602.33	2-hop traversal + access checks 325.454 ms / request	80.0%
WeBridge, Jac-like Java	synthesis failed	synthesis failed	Stage receipt	—
Original SQL Java, direct	81.53	125.95	relational lx tweets select 81.375 ms / request	98.5%
WeBridge, original SQL Java	81.10	102.76	CALL: complete outer response 81.863 ms / request	99.3%

Category costs are sums within each request; their means add to the mean total. Primary medians are computed from complete per-request sums. A CALL includes its full response and internal execution.

Source: profiles/primary-rtt-attribution/final/attribution.json; exact SQL hashes, per-request reconciliation and classifier source hashes retained.

Fresh RTT attribution: littleX mixed, 15% writes

Primary medians by delay; dominant SQL category measured at zero added delay

Configuration	Median 0 ms	Median +10 ms	Dominant category: mean RTT	Share of mean total
Jac unoptimized	4.85	200.38	acl root permissions 1.118 ms / request	23.6%
Jac optimized	4.54	154.89	acl root permissions 1.148 ms / request	25.8%
Jac-like Java, direct	188.45	407.20	Graph SELECT 148.824 ms / request	88.5%
WeBridge, Jac-like Java	synthesis failed	synthesis failed	Stage receipt	—
Original SQL Java, direct	34.49	89.11	relational lx tweets select 53.555 ms / request	93.1%
WeBridge, original SQL Java	34.24	68.96	CALL: complete outer response 29.382 ms / request	98.7%

Category costs are sums within each request; their means add to the mean total. Primary medians are computed from complete per-request sums. A CALL includes its full response and internal execution.

Source: profiles/primary-rtt-attribution/final/attribution.json; exact SQL hashes, per-request reconciliation and classifier source hashes retained.

TPC-C: statement batching and request fusion have different boundaries

Earlier warm-path SQL profiles; actual same-request RTT counts appear in the primary tables

Observed work	Native Jac original / final	Conventional Java / accepted WeBridge
NewOrder Item + Stock reads	One hydration statement binds all 10 or 30 IDs; six hydration statements total per successful request.	Three NewOrder compiler attempts fail during synthesis; no accepted generated path.
Payment object writes	One anchor batch upsert contains Warehouse, District, Customer, History and graph edge; one separate type-registry write.	Four business write statements remain four SQL statements inside the accepted good-credit procedure.
Complete good-credit Payment	Representative by-ID input: 10 SQL / 10 wire cycles; last-name input: 12 / 12. Reads, batch flush and transaction controls remain separate.	Nine steady direct wire commands become one CALL; three reads, four writes and transaction controls execute inside.
Remaining generated coverage	Native state and result gates pass for all seven latency classes.	Bad-credit and NewOrder compiler failures retain stage evidence. Whole900 conventional procedure use is separately state-rejected.

Source: native/protocol-summary.json; native/tpcc-census/SQL-map.json; tpcc-prefetch-verification.json; profiles/tpcc-final-payment-name-GC-sp-1792543948055566958/analysis.json

littleX: which SQL combines, and which stays separate

Earlier t20 three-request SQL profiles; accepted procedures retain their business SQL

Observed work	Native Jac original → final	Conventional Java → WeBridge
Feed graph traversal	Two one-hop projections + one two-hop projection: 3 → 3 SQL.	One set-oriented feed SELECT stays one business SELECT inside CALL.
Feed framework reads	Ancestry (2), root permissions (3), guest identity (2): 7 → 7. Hydration: 2 → 1.	Conventional form directly reads business tables; its result and committed state match the source model.
Feed transaction controls	6 → 3 controls; with hydration reduction, 18 → 14 complete cycles.	5 → 3 cycles. CALL contains autocommit controls and SELECT; two setup commands stay outside.
Like / create_tweet writes	One anchor batch upsert + one type-registry statement remain separate from reads and transaction controls.	UPDATE + SELECT / INSERT + SELECT remain two business statements inside CALL; 6 → 3 wire cycles.
Follow write	One anchor upsert + one registry statement remain separate SQL.	One INSERT IGNORE moves inside CALL; 5 → 3 wire cycles.

Source: native/littlex-cycle-evidence.json; profiles/littlex-batch-1792547993708251332; profiles/littlex-batch-1792547997581462562; business-verification.json in each profile

Profiling separates fewer round trips from faster SQL

Secondary rich-trace diagnostic: 85 actual feed_page requests in the 15% write schedule; +2 ms response delay

Configuration	Summed RTT ms	Exchanges / request	Server phase / statement sum ms
Jac original	62.9308	18	3.8300
Jac optimized	49.6738	14	3.7160
Original SQL Java	45.9708	5	34.2933
WeBridge, original SQL Java	41.1059	3	33.9501
Pruned Java	191.3729	7	174.3429
WeBridge, pruned Java	182.5899	3	174.8731

Source: profiles/paged-feed-cost-analysis/paged-feed-w15-subset.json; EXPLANATION.md; exact wire, PostgreSQL phase and MySQL P_S event joins

The remaining feed SELECT accounts for measured server work

Same secondary +2 ms rich-trace diagnostic; conventional Java and its generated procedure

Conventional Java executes one business SELECT per feed page. WeBridge moves that SELECT into CALL and reduces five complete client exchanges to three.

The SELECT takes a median 34.1725 ms directly and 33.7765 ms inside the procedure; it returns 20 rows and examines a median 20,110 rows. Two setup commands remain outside CALL.

The pruned graph form retains three wrapped top-K SELECTs: seven client exchanges become three, while inclusive server work remains about 174 ms. The profile distinguishes query work from command bundling.

Source: profiles/paged-feed-cost-analysis/EXPLANATION.md; mysql-request-rows.json; SELECT event IDs and generated SQL in the retained profiles

A set-oriented feed already uses one business SELECT

littleX full feed, 20 tweets/user; all 1,020 returned rows and committed state verified

Measured operation	Direct Java	Generated procedure path
External transaction/session settings	4	2
External business SELECT	1	0
External CALL	0	1
Total complete wire responses	5	3
Inside CALL: transaction settings	—	2
Inside CALL: business SELECT	—	1
Inside CALL: result materialization / metadata	—	1

Source: profiles/littlex-batch-1792547993708251332/t20-feed-LittleXSql-baseline/analysis.json; profiles/littlex-batch-1792547997581462562/t20-feed-LittleXSql-sp/analysis.json

littleX: which business SQL actually runs inside CALL

Final conventional Java; three measured wire profiles per operation, with full result and state gates

Operation	Direct wire cycles	WB wire cycles	Business SQL inside CALL
Full feed (t20)	5	3	1 feed SELECT
Like (t20)	6	3	1 tweet UPDATE + 1 tweet SELECT
Follow (t20)	5	3	1 INSERT IGNORE into follows
Create tweet (t20)	6	3	1 tweet INSERT + 1 tweet SELECT
Feed page (t200)	5	3	1 page SELECT

Source: profiles/littlex-batch-1792547993708251332 and littlex-batch-1792547997581462562; per-operation analysis.json + business-verification.json

A complete mixed schedule executes all generated APIs

littleX W3, 15% writes; 110 warmup + 100 measured requests; verified complete-bundle deployment

Operation	Requests	Direct cycles	WB cycles	Procedure active
create_tweet	10	6	3	10
feed_page	85	5	3	85
follow	1	5	3	1
like	4	6	3	4

Source: profiles/littlex-batch-1792549904262819171 and littlex-batch-1792550987625646491; protocol + P_S + independent state gates

Pruned mixed workload: fusion follows the available regions

Complete t200 15% write schedule: 110 warmup + 100 measured requests; all responses and final common state pass

Operation	Count	Direct wire cycles	WeBridge path and cycles
Paged feed	85	7	85 call-only requests; 3 cycles each.
Create Tweet	10	6	10 CALL + source-SQL requests; 6 cycles. Actual ROLLBACK TO fails, then source upsert completes.
Like	4	7	No generated region; 4 source fallbacks, 7 cycles each.
Follow	1	8	No generated region; 1 source fallback, 8 cycles.
Whole measured schedule	100	0 CALLs	95 requests issue CALL; 85 finish with SDK enabled. These record different execution outcomes.

Source: profiles/littlex-batch-1792555546513752376; profiles/littlex-batch-1792555553389885692; profiles/finish-mixed-1792555750399490880; per-request analysis.json and business-verification.json

Full Java alignment exposes the transaction-control difference

Earlier t20 three-request warm traces; identical business rows, measured transaction-control differences

Executed SQL category	Jac original / PG	Full Java / MySQL	Jac final / PG
Transaction controls	6	8	3
Full-anchor hydration	2	2	1
Ancestry + root permissions	5	5	5
Guest identity reads	2	2	2
Projected traversal stages	3	3	3
Total commands / wire cycles	18	20	14

Source: native/littlex-cycle-evidence.json; profiles/littlex-batch-1792549879927774846; final SQL text and category audits

Jac TPC-C already batches NewOrder's loop reads

Earlier original / final protocol census; equal base SQL and synchronization counts

Request class	Observed warm SQL / synchronization cycles	Mechanism
Payment by last name, GC / BC	12 / 12 and 11 / 11	Customer selection, framework reads, flush and commit remain visible
Payment by ID, GC / BC	10 / 10 and 10 / 10	Same base count in original and final
NewOrder, 5 or 15 lines	14 / 14 for either size	Original compiler already prefetches Items and Stocks across the loop
Invalid-item rollback	17 SQL / 18 complete exchanges	One separate Parse + Sync exchange is observed in the request trace

Source: native/protocol-summary.json; original pyast_gen_pass.impl.jac prefetch lowering; native source audit; tpcc-prefetch-verification.json (24 raw request logs)

SQL traces identify the deferred-setup reduction

Original ad469 versus final32 defaults; earlier census, source audit and pilot request distributions

Operation	Early graph-backend SQL: original → final	Measured / source-matched change
Full feed / original paged feed	16 → 12	Graph setup removes global-root hydration and SERIALIZABLE prologue. Base all-DB feed count includes2 guest identity reads:18 →14.
Like	11 → 10	Deferred global-root initialization
Follow	12 → 11	Deferred global-root initialization
Create tweet	9 → 8	Deferred global-root initialization
Business read traversal	Same 3 projected query stages in both	Root → Profile; own tweets; Follow → Profile → tweets. Both retain ancestry and access checks.
LIKE: 3 post-warmup targets	[10,11,11] → [9,10,10]	First target owns the Tweet: permission check returns before owner-root hydration. Other two targets have different owners.
Periodic schema guard	+1 in some t100/t200 profiles	Actual to_regclass table-presence check; all-DB ranges18–19 →14–15 retain this extra request.

Source: native corrected littleX SQL censuses; context.impl.jac source audit; runtime-provenance.json; native/littlex-pilot-like/certified-requests.json; native/littlex-like-alignment.json

COMMIT cost varies even when SQL and wire counts stay fixed

Separate +2 ms ID-Payment diagnostic; 30 measured requests per cell

Measured quantity	Original GC / BC	Final GC / BC
SQL / complete response cycles	10 / 10 for every measured request	10 / 10 for every measured request
PostgreSQL COMMIT execution median	13.467 ms / 1.288 ms	1.566 ms / 1.620 ms
Later WAL-instrumented original probe	GC: 1.689 ms; BC: 1.626 ms	23 sampled COMMIT WalSync waits in each original class
Interpretation	Observed slow COMMIT is inside PostgreSQL execution	Later probe observes normal commit WALSync work; earlier outlier attribution stays open

Source: native/tpcc-outlier-diagnostic/analysis.json; native/tpcc-wal-diagnostic/WAL-analysis.json

A longer +10 ms trace checks NewOrder15 over time

Secondary diagnostic: 10 warmup + 60 measured requests per runtime

All 120 measured requests execute 14 SQL commands and 14 complete delivered-response exchanges. The trace spans more than five seconds per runtime.

This class-specific diagnostic retains its complete timeline and complements the primary compact RTT collections at 0 / +2 / +10 ms.

Source: native/tpcc-ttl-diagnostic/certified-requests.json; native/tpcc-ttl-diagnostic/count-distribution.json

Source-faithful money handling exposes a numerical model boundary

Observed NewOrder synthesis failures and bad-credit Payment installation failures

The Java port preserves Python's round(float, 2) through BigDecimal(double) and HALF_EVEN. Native symbolic replay reaches scales 45–47 before setScale(2).

The symbolic model forms a long divisor from pow(10, scale difference); the observed expression contains -9223372036854775808. The source model also leaves the rounding-mode argument unused.

NewOrder's emitted symbolic expression fails Java-expression parsing. Bad-credit Payment emits the expression as a procedure formal parameter and MySQL rejects installation. Exact stage artifacts identify each affected case.

Source: tpcc/numeric-model-boundary.json; tpcc/stage-failures.json; case-linked original symbolicBigDecimal.cpp and SPParser diagnostics

Whole-plan references record actual procedure coverage

Each configuration executes the original 900 requests; guarded fallback and source-only paths are counted separately

WeBridge configuration	CALL requests	SDK enabled at end	Guard fallback	Source-only requests
Full graph	0	0	0	900
Pruned graph	0	0	0	900
Conventional / String IDs	406	130	276	494
Conventional / typed IDs control	242	64	178	658

Source: tpcc/reference-900-index.json; tpcc/mixed-results.json; typed control routing, raw request and state receipts

Whole-plan correctness catches a late-fallback state error

Tested JDBC getString → Integer.parseInt port; all900 requests finish, committed-state gate rejects the run

Observed gate	Result	Evidence
Returned business values	All900 equal the original direct execution	Per-request source-golden comparison
History inserts	586 observed;406 required	Full eight-relation state comparison
First divergent request: idx5	Expected customer2700; generated path uses270	Captured customer329 used three character positions in emitted integer conversion
Exact first-fallback state	Both customer270 and2700 debit2432.68;4History rows for3Payments	Fresh original six-request prefix; warehouse total equals3Payments exactly
Protocol and server order	Procedure writes + SET autocommit=1; then failed ROLLBACK TO; then source customer/history writes	Complete CALL events and wire responses; tested SDK and session configuration retained

Source: tpcc/mixed-results.json; tpcc/mixed-conventional-full-20261021T013535Z/correctness-failure.json; tpcc/mixed-first-fallback-prefix6/observed-state.json

Typed-ID control exposes a result-cardinality boundary

Same business SQL with JDBC getInt customer IDs; original ordered workload and actual protocol evidence

Observed boundary	Measured result
Typed prefix0..5	All six outputs and eight initial/final business relations match fresh direct execution.
Typed original900	All 900 outputs match; committed History adds 488 rows for 406 Payments.
First changed result shape	Original index22: emitted customer materialization caps at two rows; the live SELECT returns three.
Stored-program order	All 21 nested events captured. SET autocommit=1 (event877) precedes final metadata SELECT (event878).
Actual fallback	ROLLBACK TO trace0_p4 fails. Direct customer SELECT and Warehouse / District / Customer / History writes follow outside CALL.
Exact prefix23 state gap	One extra 1,782.87 Payment in Warehouse / District / Customer(1,1,1103); History adds 11 rows for 10 Payments. Direct model passes; independent model rejects SP state.

Source: tpcc/controls/typed-customer-id/reference-20261021T024832Z; profiles/tpcc-typed-fallback-prefix23-17925516693998538 22-sp/analysis.json; tpcc/controls/typed-customer-id/first-fallback-prefix23/observed-state.json

What the evidence bundle preserves

Re-run scripts, per-request observations, independent oracles and source identities

Every reported cell points to raw request timings or a concrete compiler-stage failure, plus its fixture reset, effective settings and correctness receipts.

SQL/protocol profiles retain command text, complete response ordering, actual procedure activation, server statement events and explicit category totals.

Large identical state payloads share one gzip object through relative links. The archive ledger verifies the original uncompressed bytes, stored object hashes and each logical evidence path.

Source: PUBLIC_FILES manifests; archive-manifest.json; verify_archive.py; results.json

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
TPC-C payment name GC	Java direct	36.75	55.12	124.77	8 / 8 / 8
	WeBridge	replay failed	replay failed	replay failed	stage failure
TPC-C new order rollback	Java direct	32.84	62.08	156.33	12 / 12 / 12
	WeBridge	replay failed	replay failed	replay failed	stage failure
TPC-C payment id GC	Java direct	6.08	23.55	79.18	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	stage failure
TPC-C new order 5	Java direct	16.60	38.30	107.88	9 / 9 / 9
	WeBridge	synthesis failed	synthesis failed	synthesis failed	stage failure
TPC-C new order 15	Java direct	31.32	50.04	121.21	9 / 9 / 9
	WeBridge	synthesis failed	synthesis failed	synthesis failed	stage failure

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
TPC-C payment name BC	Java direct	33.13	48.39	108.22	7 / 7 / 7
	WeBridge	replay failed	replay failed	replay failed	stage failure
TPC-C payment id BC	Java direct	5.87	22.33	80.32	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	stage failure
littleX feed t20	Java direct	57.00	69.14	139.14	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	synth
littleX like t20	Java direct	13.85	30.07	87.39	7 / 7 / 7
	WeBridge	replay failed	replay failed	replay failed	replay
littleX follow t20	Java direct	23.07	41.18	110.60	8 / 8 / 8
	WeBridge	replay failed	replay failed	replay failed	replay

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
littleX create tweet t20	Java direct	13.97	27.24	77.55	6 / 6 / 6
	WeBridge	16.88*	31.17*	83.42*	6 / 6 / 6
littleX feed t50	Java direct	112.31	124.39	186.29	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	synth
littleX like t50	Java direct	18.30	34.31	100.14	7 / 7 / 7
	WeBridge	replay failed	replay failed	replay failed	replay
littleX follow t50	Java direct	31.62	49.00	124.14	8 / 8 / 8
	WeBridge	replay failed	replay failed	replay failed	replay
littleX create tweet t50	Java direct	18.07	34.03	87.83	6 / 6 / 6
	WeBridge	20.95*	35.04*	90.54*	6 / 6 / 6

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
littleX feed t100	Java direct	205.01	226.75	270.65	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	synth
littleX like t100	Java direct	25.11	40.47	104.08	7 / 7 / 7
	WeBridge	replay failed	replay failed	replay failed	replay
littleX follow t100	Java direct	44.80	62.63	136.92	8 / 8 / 8
	WeBridge	replay failed	replay failed	replay failed	replay
littleX create tweet t100	Java direct	24.96	38.06	97.03	6 / 6 / 6
	WeBridge	27.83*	41.47*	101.79*	6 / 6 / 6
littleX feed t200	Java direct	380.96	396.84	441.46	7 / 7 / 7
	WeBridge	synthesis failed	synthesis failed	synthesis failed	synth

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
littleX like t200	Java direct	39.19	53.30	117.45	7 / 7 / 7
	WeBridge	replay failed	replay failed	replay failed	replay
littleX follow t200	Java direct	72.70	90.18	161.61	8 / 8 / 8
	WeBridge	replay failed	replay failed	replay failed	replay
littleX create tweet t200	Java direct	39.00	52.47	111.84	6 / 6 / 6
	WeBridge	41.97*	55.30*	112.18*	6 / 6 / 6
littleX mixed t200 / 0% writes	Java direct	167.57	184.28	248.47	7 / 7 / 7
	WeBridge	167.68	173.82	204.37	3 / 3 / 3
littleX mixed t200 / 5% writes	Java direct	167.86	184.63	248.93	6–8 / 6–8 / 6–8
	WeBridge	166.79*	175.19*	203.24*	3–8 / 3–8 / 3–8

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Supplementary comparison: pruned Java and WeBridge

Summed SQL RTT per request (ms); command counts use the same requests

Case	Pruned form	0 ms	+2 ms	+10 ms	SQL commands: 0 / +2 / +10
littleX mixed t200 / 15% writes	Java direct	167.97	183.56	250.12	6–8 / 6–8 / 6–8
	WeBridge	166.74*	176.61*	203.82*	3–8 / 3–8 / 3–8
littleX mixed t200 / 30% writes	Java direct	167.81	184.97	244.93	6–8 / 6–8 / 6–8
	WeBridge	165.57*	175.48*	204.71*	3–8 / 3–8 / 3–8

* Includes source fallback or incomplete procedure use; exact CALL / SDK activation fractions are retained per request. Parentheses in count cells show exchanges when different from SQL commands.

Source: results.json / results.csv; per-case raw RTT, command-count and correctness links retained

Original TPC-C plan: DB RTT and SQL command counts

Original 900 ordered transactions: 100 warmup + 800 measured; medians and SQL ranges use the measured 800

Configuration	Median RTT / SQL range	CALLs / 900	Result / final state
Jac unoptimized	8.88 ms 10–17 SQL	0	PASS
Jac optimized	5.65 ms 10–17 SQL	0	PASS
Jac-like Java, direct	14.74 ms 10–17 SQL	0	PASS
WeBridge, Jac-like Java	12.01 ms 10–17 SQL	0	PASS: source fallback
Java pruned	10.25 ms 7–12 SQL	0	PASS
WeBridge pruned	11.50 ms 7–12 SQL	0	PASS: source fallback
Original SQL Java, direct	7.51 ms 9–68 SQL	0	PASS
WeBridge, original SQL Java	state rejected	406	State rejected

Source: rtt-reference-results.json. Actual CALL totals cover all 900 requests; source-fallback paths and rejected state retain their raw evidence.

Original littleX references: 20 tweets per user

Pure operations: 110 warmup + 500 measured; each cell shows median summed RTT and per-request SQL range

Configuration	Full feed	Like	Follow	Create tweet
Jac unoptimized	13.43 ms 18–19 SQL	2.91 ms 10–11 SQL	3.59 ms 12 SQL	2.69 ms 9 SQL
Jac optimized	12.79 ms 14–15 SQL	2.73 ms 9–10 SQL	3.35 ms 11 SQL	2.58 ms 8 SQL
Jac-like Java, direct	71.10 ms 20 SQL	14.92 ms 11–12 SQL	25.19 ms 13 SQL	15.14 ms 10 SQL
WeBridge, Jac-like Java	compiler stage failed	compiler stage failed	compiler stage failed	compiler stage failed
Original SQL Java, direct	9.42 ms 5 SQL	3.23 ms 6 SQL	3.02 ms 5 SQL	3.26 ms 6 SQL
WeBridge, original SQL Java	9.04 ms 3 SQL	2.96 ms 3 SQL	2.80 ms 3 SQL	3.06 ms 3 SQL

Source: rtt-reference-results.json; full source results and evolving committed state are independently checked.

Original littleX references: 50 tweets per user

Pure operations: 110 warmup + 500 measured; each cell shows median summed RTT and per-request SQL range

Configuration	Full feed	Like	Follow	Create tweet
Jac unoptimized	21.37 ms 18–19 SQL	2.85 ms 10–11 SQL	3.54 ms 12 SQL	2.71 ms 9 SQL
Jac optimized	20.48 ms 14–15 SQL	2.79 ms 9–10 SQL	3.41 ms 11 SQL	2.54 ms 8 SQL
Jac-like Java, direct	125.23 ms 20 SQL	18.89 ms 11–12 SQL	32.79 ms 13 SQL	19.75 ms 10 SQL
WeBridge, Jac-like Java	compiler stage failed	compiler stage failed	compiler stage failed	compiler stage failed
Original SQL Java, direct	21.17 ms 5 SQL	3.22 ms 6 SQL	2.91 ms 5 SQL	3.44 ms 6 SQL
WeBridge, original SQL Java	20.46 ms 3 SQL	2.97 ms 3 SQL	2.91 ms 3 SQL	2.89 ms 3 SQL

Source: rtt-reference-results.json; full source results and evolving committed state are independently checked.

Original littleX references: 100 tweets per user

Pure operations: 110 warmup + 500 measured; each cell shows median summed RTT and per-request SQL range

Configuration	Full feed	Like	Follow	Create tweet
Jac unoptimized	34.13 ms 18–19 SQL	2.90 ms 10–11 SQL	3.61 ms 12 SQL	2.70 ms 9 SQL
Jac optimized	33.79 ms 14–15 SQL	2.97 ms 9–10 SQL	3.54 ms 11 SQL	2.55 ms 8 SQL
Jac-like Java, direct	215.66 ms 20 SQL	26.55 ms 11–12 SQL	46.53 ms 13 SQL	27.00 ms 10 SQL
WeBridge, Jac-like Java	compiler stage failed	compiler stage failed	compiler stage failed	compiler stage failed
Original SQL Java, direct	40.83 ms 5 SQL	3.05 ms 6 SQL	2.98 ms 5 SQL	3.28 ms 6 SQL
WeBridge, original SQL Java	37.28 ms 3 SQL	2.99 ms 3 SQL	2.81 ms 3 SQL	2.93 ms 3 SQL

Source: rtt-reference-results.json; full source results and evolving committed state are independently checked.

Original littleX references: 200 tweets per user

Pure operations: 110 warmup + 500 measured; each cell shows median summed RTT and per-request SQL range

Configuration	Full feed	Like	Follow	Create tweet
Jac unoptimized	63.32 ms 18–19 SQL	3.00 ms 10–11 SQL	3.76 ms 12 SQL	2.76 ms 9 SQL
Jac optimized	62.33 ms 14–15 SQL	2.87 ms 9–10 SQL	3.54 ms 11 SQL	2.74 ms 8 SQL
Jac-like Java, direct	398.39 ms 20 SQL	40.00 ms 11–12 SQL	74.84 ms 13 SQL	40.79 ms 10 SQL
WeBridge, Jac-like Java	compiler stage failed	compiler stage failed	compiler stage failed	compiler stage failed
Original SQL Java, direct	79.39 ms 5 SQL	3.25 ms 6 SQL	3.19 ms 5 SQL	3.20 ms 6 SQL
WeBridge, original SQL Java	77.83 ms 3 SQL	3.01 ms 3 SQL	2.84 ms 3 SQL	5.19 ms 3 SQL

Source: rtt-reference-results.json; full source results and evolving committed state are independently checked.

Original littleX mixed references: 200 tweets per user

110 warmup + 1,000 measured; median summed RTT and per-request SQL range over the complete measured schedule

Configuration	0% writes	5% writes	15% writes	30% writes
Jac unoptimized	5.17 ms 18–19 SQL	4.99 ms 9–19 SQL	5.25 ms 9–19 SQL	4.72 ms 9–19 SQL
Jac optimized	4.69 ms 14–15 SQL	4.69 ms 8–15 SQL	4.50 ms 8–15 SQL	4.50 ms 8–15 SQL
Jac-like Java, direct	188.64 ms 20 SQL	187.84 ms 10–20 SQL	191.57 ms 10–20 SQL	188.12 ms 10–20 SQL
WeBridge, Jac-like Java	compiler stage failed	compiler stage failed	compiler stage failed	compiler stage failed
Original SQL Java, direct	34.05 ms 5 SQL	34.33 ms 5–6 SQL	34.21 ms 5–6 SQL	34.20 ms 5–6 SQL
WeBridge, original SQL Java	34.08 ms 3 SQL	33.99 ms 3 SQL	34.05 ms 3 SQL	34.29 ms 3 SQL

Source: rtt-reference-results.json; exact CALL / SDK activation and per-operation measurements remain in raw request records.